PharmaSoft | Informe técnico de arquitectura

Lenguaje de Programación II · Actividad autónoma de la sesión 7 · 11 de octubre de 2026. Autor: pendiente de completar con el nombre del estudiante.

Alcance: revisión y adaptación del módulo Clientes existente en tarea07-frontend, con compatibilidad del backend local. Este informe describe el código entregado; las evidencias de navegador y la entrega Git se identifican como pendientes cuando no se han ejecutado.

B1. Estructura del proyecto

```
src/app/
  app.ts, app.html, app.css, app.config.ts, app.routes.ts
  core/
    config/menu.ts
    models/{error-response,pagina-response}.ts
    utils/{http-error,numeros,texto}.ts
  shared/pages/no-encontrado/ [ts, html, css]
  layout/{header,sidebar,main-layout}/ [ts, html, css]
  features/
    inicio/ [ts, html, css]
    categorias/
      categorias.routes.ts
      models/categoria.model.ts
      services/categoria-service.ts
      pages/{categoria-list,categoria-form}/ [ts, html, css]
    clientes/
      clientes.routes.ts
      models/cliente.model.ts
      services/cliente-service.ts
      pages/{cliente-list,cliente-form}/ [ts, html]
    productos/
      productos.routes.ts
      models/producto.model.ts
      services/producto-service.ts
      pages/{producto-list,producto-form}/ [ts, html]
```

Árbol agrupado de archivos de producción. El árbol completo, incluidos los archivos .spec.ts, está en docs/arbol-src-app.txt. Productos existe en disco, pero no está registrado en las rutas principales ni en MENU.

core reúne configuración, contratos y utilidades transversales; shared contiene la página reutilizable de error 404; layout mantiene la estructura visual; features agrupa cada dominio con modelos, servicio, páginas y rutas. La paginación se comparte mediante PaginaResponse<T>, mientras que las reglas de Clientes permanecen en su feature.

```
// src/app/features/clientes/services/cliente-service.ts:11
private readonly url = `${environment.apiUrl}/clientes`;
// src/app/features/clientes/pages/cliente-form/cliente-form.ts:46
const dto: ClienteRequest = { ...v, telefono: v.telefono || null, direccion: v.direccion || null };
```

B2. Mapa completo de rutas activas

URL	Componente / padre	Carga	Título
/	Redirección a /inicio / MainLayout	Redirección	Hereda destino
/inicio	Inicio / MainLayout	loadComponent	PharmaSoft
/categorias	CategoriaList / MainLayout	loadChildren → component	Categorías
/categorias/nuevo	CategoriaForm / MainLayout	loadChildren → component	Nueva categoría
/categorias/:id/editar	CategoriaForm / MainLayout	loadChildren → component	Editar categoría
/clientes	ClienteList / MainLayout	loadChildren → component	Clientes
/clientes/nuevo	ClienteForm / MainLayout	loadChildren → component	Nuevo cliente
/clientes/:id/editar	ClienteForm / MainLayout	loadChildren → component	Editar cliente
/**	NoEncontrado / sin padre	loadComponent	Página no encontrada

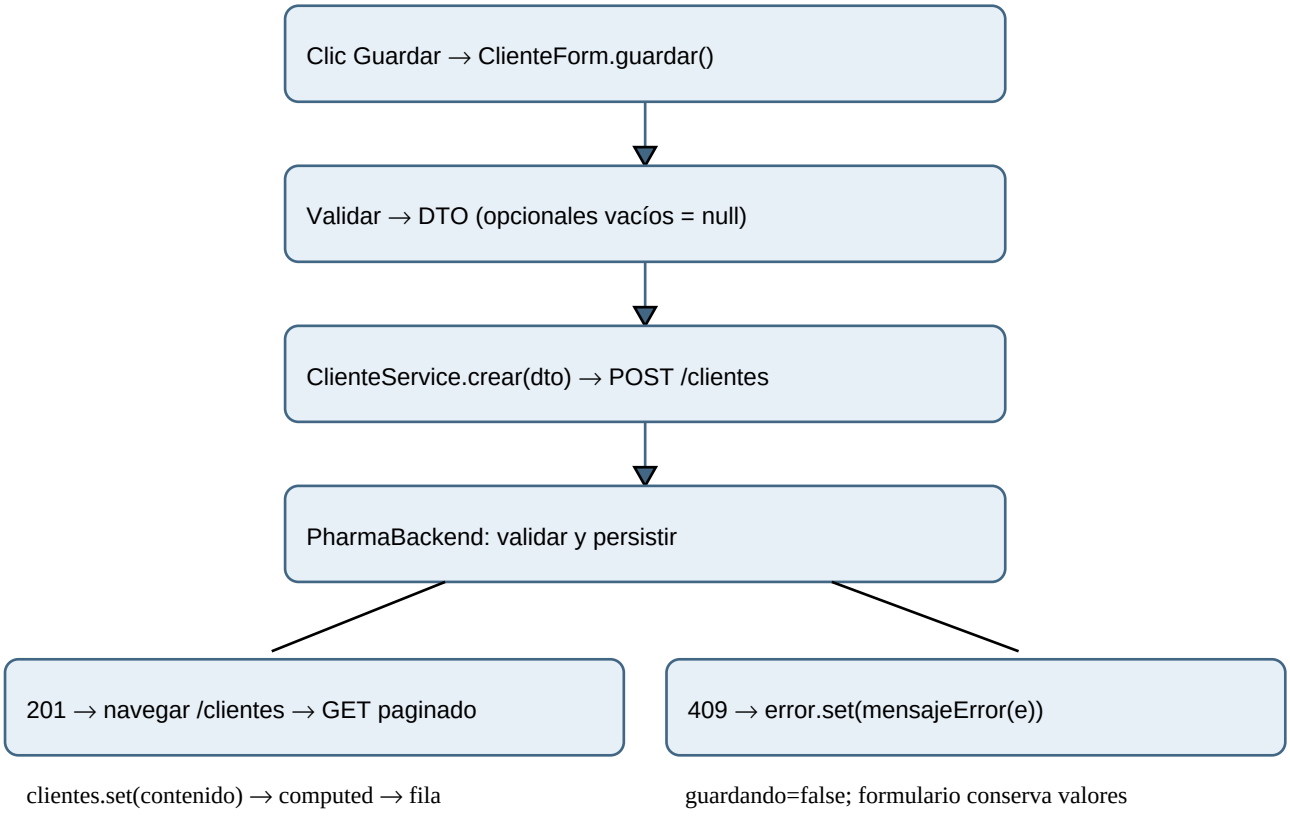
Fuentes: src/app/app.routes.ts:3; src/app/features/clientes/clientes.routes.ts:5; src/app/features/categorias/categorias.routes.ts:5. MainLayout se importa directamente; los componentes de cada feature se importan desde su archivo de rutas diferido.

B3. Responsabilidades y límites

Pieza	Qué hace	Qué no hace
ClienteService / servicio	Cinco operaciones HTTP tipadas, parámetros de página.	No presenta mensajes ni navega.
ClienteList / componente	Signals, filtro computed, orden, página y baja.	No construye HTTP ni valida unicidad.
ClienteForm / componente	Validación, carga por ID, DTO, guardar y navegar.	No decide unicidad ni persiste directamente.
CategoriaService / servicio	Encapsula HTTP de Categorías.	No controla la vista.
CategoriaList / componente	Carga, búsqueda local y eliminación de categorías.	No conoce URL ni consulta la base de datos.
CategoriaForm / componente	Formulario reactivo, edición y errores.	No implementa reglas de integridad del servidor.
MainLayout / componente	Header, Sidebar, outlet y menú colapsado.	No administra entidades.
Header / componente	Marca, enlaces y evento toggleMenu.	No consulta clientes.
Sidebar / componente	Recorre MENU y marca ruta activa.	No realiza CRUD ni autentica usuarios.

B4. Flujo de registrar cliente

El diagrama sigue las llamadas reales del formulario y del listado; el signal de clientes se actualiza después del GET posterior a la navegación, no con el POST directamente.



Validación local fallida: markAllAsTouched() y retorno antes de invocar al servicio. Un 409 muestra el mensaje del backend, mantiene los datos y permite corregirlos. Un 400 puede mostrar además erroresServidor por campo.

```
// src/app/features/clientes/pages/cliente-form/cliente-form.ts:49
request.subscribe({
  next: () => this.router.navigate(['/clientes'], { queryParams: { guardado: 1 } }),
  error: e => { this.guardando.set(false); this.error.set(mensajeError(e)); /* ... */ }
});
```

Fuentes adicionales: src/app/features/clientes/pages/cliente-list/cliente-list.ts:31 (contenido y metadatos), src/app/features/clientes/pages/cliente-list/cliente-list.ts:24 (computed) y src/app/features/clientes/services/cliente-service.ts:19 (POST).

Compatibilidad del backend entregado

La copia local anterior devolvía Iterable y eliminaba físicamente. Se adaptaron ClienteController.java, ClienteService.java y ClienteServiceImpl.java: GET paginado, orden limitado a id/dni/nombres/apellidos/email, parámetros inválidos con 409, baja estado=false y 409 al repetirla. La consulta conserva clientes inactivos y agrega id como desempate del orden. Categorías no se modificó.

B5. Preguntas de revisión

1. Archivos modificados y arquitectura

En esta sesión se modificaron tres archivos existentes del frontend: cliente-service.ts, cliente-list.ts y cliente-list.html. Clientes, sus modelos, sus rutas y MENU ya existían. Para agregar el feature desde la base de Categorías, los dos puntos transversales son app.routes.ts y core/config/menu.ts; no se afirma que se hayan modificado ahora. Además se adaptaron tres archivos del backend. La incorporación está localizada en el dominio y no exige cambiar Header o Sidebar. src/app/app.routes.ts:19; src/app/core/config/menu.ts:9.

2. Por qué el listado no usa HttpClient

El listado coordina presentación y estados y depende de ClienteService. La URL base está en environment.apiUrl y el recurso en el servicio; cambiar el servidor exige ajustar el environment correspondiente, sin editar el listado. src/app/features/clientes/services/cliente-service.ts:11; src/app/features/clientes/pages/cliente-list/cliente-list.ts:9.

3. Ventaja concreta de loadChildren

El archivo de rutas y sus imports se descargan cuando se necesita Clientes. app.routes.ts no importa sus páginas directamente; el límite del feature facilita crecimiento y división del bundle. ng build completado: chunk-BMcrz_Lb.js | clientes-routes | 10.43 kB | 3.38 kB estimados de transferencia. Esta salida verifica un chunk diferido del feature; Categorías conserva su chunk independiente. Registro: docs/validacion-build.txt. src/app/app.routes.ts:19.

4. Persistencia del layout

Categorías y Clientes son rutas hijas del mismo MainLayout. El RouterOutlet interno cambia de página y conserva la instancia padre con Header y Sidebar. Puede haber detección de cambios, pero no una recreación del layout por cada navegación entre esas hijas. src/app/layout/main-layout/main-layout.html:6; src/app/app.routes.ts:7.

5. Ocultar Clientes por permisos

El punto de definición es core/config/menu.ts. Allí se añadiría el permiso requerido y el Sidebar filtraría los elementos según el usuario; el Sidebar actual solo recorre MENU y todavía no implementa ese filtro. Centralizar los datos evita duplicar decisiones por página, pero ocultar el enlace no protege la URL: también hacen falta guards y autorización en backend. src/app/core/config/menu.ts:6; src/app/layout/sidebar/sidebar.html:4.

6. Duplicación y propuesta

Los formularios repiten estados de carga, guardado, errores y navegación; los listados repiten búsqueda y presentación de errores. Se propone un componente de alerta y controles de paginación en shared, conservando validadores y DTO por dominio. No conviene un CRUD genérico que confunda baja lógica de Clientes y eliminación de Categorías. src/app/features/clientes/pages/cliente-form/cliente-form.ts:16; src/app/features/categorias/pages/categoria-form/categoria-form.ts:22.

7. Búsqueda limitada a la página

filtrados depende de clientes(), que contiene únicamente respuesta.contenido. El computed no realiza peticiones. Para buscar globalmente, el backend debe aceptar un parámetro de búsqueda y filtrar en la consulta antes de paginar y contar; ClienteService lo enviaría mediante HttpParams. El componente reiniciaría la página y aplicaría debounce y cancelación de solicitudes. src/app/features/clientes/pages/cliente-list/cliente-list.ts:24; src/app/features/clientes/services/cliente-service.ts:14.

A7. Verificación y evidencias de entrega

Estado: no hay PharmaBackend escuchando en localhost:8080 al revisar el entorno. Los siguientes son casos de aceptación pendientes de ejecución integrada y de captura con la pestaña Red visible. Esta tabla no reemplaza las once capturas exigidas. La compilación y pruebas disponibles se registran por separado en docs/.

N.º	Acción / evidencia que debe guardarse
1	Abrir Clientes desde sidebar: activo y GET 200 con pagina=0.
2	Tamaño 5 y Siguiente: tamanio=5, pagina=1; Página 2.
3	Apellidos dos veces: comprobar asc y desc y cambio de orden.
4	DNI de 7 dígitos: validación visible, sin POST.
5	Alta válida sin opcionales: POST 201, null y teléfono como guion.
6	DNI o correo duplicado: POST 409 y mensaje del backend.
7	Editar teléfono: GET precarga y PUT 200 con nueve dígitos.
8	Buscar DNI y apellido: filtro local sin nueva petición.
9	Baja activa: DELETE 204, GET posterior y estado Inactivo.
10	Repetir baja: DELETE 409 y mensaje del backend.
11	Regresar a Categorías: comprobar listado y operaciones.

C. Versionamiento y entrega: pendientes verificables

Las carpetas recibidas no contienen un repositorio Git ni remoto. No se puede acreditar una rama creada desde develop, commits históricos ni un pull request. No se generaron fechas falsas. El estudiante debe vincular este trabajo a su repositorio real, crear feature/clientes-<apellido> desde develop y registrar avances reales. El requisito de tres días distintos no puede satisfacerse retroactivamente en esta sesión.

Enlace del pull request hacia develop: pendiente de crear en el repositorio real. Nombre de entrega en aula: Apellido_LP2_S7_Autonoma.pdf, reemplazando Apellido por el del estudiante. Antes de entregar, completar autor, enlace y capturas. El chunk ya se verificó con ng build. El informe se guarda en docs/informe-arquitectura.pdf.

Cómo reproducir

Backend: Java 21, Oracle configurado en application-dev.yaml; ejecutar mvnw.cmd spring-boot:run y comprobar Swagger. Frontend: npm ci, npm run build, npm test -- --watch=false y npm start. Mantener localhost:8080 y localhost:4200 según la configuración del proxy. Guía operativa ampliada: docs/ENTREGA.md.